

PBL 3 - Sistema de Recarga de VEs Baseado em Blockchain

José Victor de Oliveira Correia, João Victor Macedo dos Santos Lima

Departamento de Tecnologia – Universidade Estadual de Feira de Santana (UEFS)
44036–900 – Feira de Santana – Bahia

josevictoroliveira160@gmail.com , joao.macedo.lima@hotmail.com

Resumo: Problema proposto com o objetivo de aprimorar um sistema de reservas e recarga para carros elétricos que precisam realizar viagens de longa distância, implementando o conceito de Blockchain e livro razão para realizar transações de diversos tipos, de forma confiável e transparente.

1. Introdução

Através do projeto “Problema 3: Sistema de Recarga de VEs Baseado em Blockchain”, foi proposto o aprimoramento do sistema de recarga existente, integrando uma solução baseada em livro razão distribuído ou blockchain. O objetivo principal é registrar de forma segura e transparente todas as transações de reserva, recarga e pagamento entre as empresas envolvidas, promovendo confiança entre usuários e empresas. Para isso, será desenvolvida uma solução descentralizada, utilizando diversas tecnologias e frameworks de terceiros, como: comunicação MQTT, arquitetura API REST, Docker, Ganache, Web3.py, linguagem Python, conceito de contratos inteligentes utilizando a linguagem Solidity e aplicativos descentralizados(dApps)

2. Fundamentação Teórica

Para o desenvolvimento desse sistema de recarga de veículos elétricos, foram utilizados fundamentos e ferramentas pontuais, segue abaixo a listagem destas:

- **Linguagem Python**

A linguagem escolhida para o desenvolvimento do sistema foi o Python, característica por ser simples, facilmente legível, por suportar a execuções em multiparadigma e utilizando POO, rica em bibliotecas nativas para o desenvolvimento de lógicas complexas e suporte, mesmo que externo para os tipos de comunicações utilizados.

- **API REST**

Conhecido como **Representational State Transfer**, é um protocolo de arquitetura utilizado em comunicações via internet entre clientes e servidores, baseado em recursos acessados por URLs e manipulados com verbo HTTP.

- **MQTT**

Conhecido como **Message Queuing Telemetry Transport** , é um protocolo de

comunicação para sistemas, dispositivos e na área da Internet das coisas (IoT) que dependem de comunicação rápida, eficiente e com baixo consumo de energia e largura de banda. Tem como características e conceitos principais:

- **Broker:** Servidor responsável por troca de mensagens entre as entidades na comunicação local. Neste projeto o broker escolhido foi o **Mosquitto**, que tem suporte na linguagem python através da biblioteca externa " **paho-mqtt** " .
- **Cliente:** São as entidades de conexão local. Se baseiam no conceito de publisher/subscriber, onde podem enviar para ou receber mensagens de outras entidades. Neste projeto, tanto veículo como os servidores se comportam como clientes, trocando mensagens entre si através do broker, para alcançar o objetivo do sistema.
- **Tópicos:** São os endereços por onde os clientes se inscrevem para receber ou publicar mensagens entre si, os tópicos são gerenciados pelo broker. No projeto, os tópicos definidos foram:

Publisher:

Tópico	Descrição
"vehicle/create_reservations/server"	Tópico utilizado para criar reservas

Subscriber:

Tópico	Descrição
"server/create_reservations/vehicle"	Tópico utilizado para receber os IDs de reservas efetuadas na blockchain
"server/contracts_addresses/vehicle"	Tópico utilizado para receber os endereços de contrato

- **M2M**

Conhecido como **Machine to Machine**, é a troca de informações em redes diretamente entre máquinas e sem a influência humana em nenhum ponto. Baseado em outros conceitos como automação, baixo consumo, comunicação em tempo real, é frequentemente atrelado ao protocolo de comunicação MQTT.

- **POO**

Utilização de programação orientada a objetos para uma aproximação com a realidade, formato entidades específicas e com as suas devidas características e funções.

- **Docker**

Plataforma nativa do Linux que tem como objetivo criar, empacotar e executar aplicações em containers simultaneamente, simulando dispositivos e sistemas próprios. Utiliza os moldes imagem e compose para realizar tal estruturação.

- **Ganache**

Ganache é um ambiente de simulação de rede local Ethereum para o desenvolvimento de aplicações utilizando blockchain privada. Com os conceitos e estruturas de contratos inteligentes e dApps não há a necessidade de interagir com redes públicas, economizando tempo e custos de transação com disponibilidade de contas pré-definidas e pré-financiadas. Tem como características principais: Confiabilidade, transparência, flexibilidade e integração com outras diversas tecnologias.

- **dApps**

As dApps são aplicações que operam em uma rede blockchain. Tem como características principais: Descentralização, código aberto e transparência. Utiliza entidades conhecidas como contratos inteligentes(em linguagem Solidity) para realizar as lógicas de integração necessárias.

- **Web3.py**

O Web3.py é uma biblioteca de Python que permite e simplifica a comunicação e interação entre as dApps com a blockchain Ethereum(Ganache). Permite interagir com contratos inteligentes, com contas pré-definidas, acessar dados, realizar transações e executar eventos.

- **Contratos inteligentes e Solidity**

São aplicações autoexecutáveis que são armazenadas executadas em uma rede blockchain, estabelecendo regras transparentes, imutáveis e diretas sobre a interação entre as entidades envolvidas. Os contratos são escritos na linguagem Solidity, criada especialmente para a implementação das estruturas e lógica dos contratos inteligentes na Ethereum. Tem como características: Ser de alto-nível, orientada a contratos(DbC ou POC), de tipagem estática, confiável, e de grande interação com outras tecnologias.

3. Metodologia, Implementação e Testes

O desenvolvimento do projeto foi executado no ambiente Laboratório de Redes e Sistemas Distribuídos - LARSID, da Instituição UEFS. Além disso, alguns momentos de teste e desenvolvimento foram realizados em momentos assíncronos e virtualmente.

Testes foram realizados nas máquinas com OS Linux, disponíveis no LARSID e também nas máquinas pessoais dos integrantes (Linux e Windows através de WSL). Dentre esses testes, os mais frequentes eram feitos para estabelecer a comunicação MQTT e via internet baseado no protocolo de arquitetura API REST entre os containers desenvolvidos.

4. Arquiteturas

Arquitetura do Veículo:

O sistema e conexão cliente do contêiner “Veículo”, implementado no arquivo “main.py” continua utilizando “mqtt” em Python, é responsável por simular o sistema de um veículo elétrico, sua conta de usuário com as informações necessárias e estabelecer comunicação com o Broker MQTT e Servidores quando necessário.

— Estrutura de arquivos do Veículo:

1. main.py (main Vehicle):

É o arquivo de início do sistema, nela há a estruturação de uma interface de interação amigável ao usuário. Através do conceito POO, os processos foram divididos em classes e os devidos objetos responsáveis pela lógica de comunicação, pela formatação e exibição de dados, preenchimento e geração de informações e o fluxo geral do sistema são utilizados neste arquivo.

Desta vez, a cada novo veículo criado há o preenchimento direto dos dados de conta, de forma aleatória com a utilização de bibliotecas Faker e random, também a definição de novas variáveis para a utilização do Web3.py, manipulação de novos arquivos(arquivos json com modelos abi de contratos inteligentes) e lógicas de funcionalidade.

Nas opções de LOGIN, há as seguintes opções: Realizar reservas, visualizar reservas, ver informações de veículo, finalizar reservas após a recarga ser concluída, voltar para o início do sistema(sem a necessidade de fazer login novamente) e finalmente a opção de encerrar o sistema por completo.

2. User.py

Classe responsável por guardar as informações do proprietário do veículo. As variáveis que a compõem são:

Nome	Descrição	Tipo
cpf	CPF do proprietário	str
name	Nome do proprietário	str
email	Email do proprietário	str
password	Senha de acesso proprietário ao sistema	str

Com exceção da variável “name”, as outras 3 são utilizadas para realizar o processo de login de usuário na “main.py”, onde a informação login pode ser identificada pelo cpf ou email e a senha por password, respectivamente.

Os dados atribuídos a essas variáveis são gerados no arquivo “main.py”, utilizando a biblioteca externa ao python, chamada de **Faker**.

3. Vehicle.py

É a classe responsável por estruturar as informações do veículo. As variáveis e métodos

existentes são:

Variáveis:

Nome	Descrição	Tipo
vid	ID do veículo.	str
owner	Proprietário do veículo.	User
licensePlate	Placa do veículo.	str
moneyCredit	Crédito do veículo.	float
currentEnergy	Energia atual do veículo.	int
maximumBattery	Capacidade máxima da bateria do veículo.	int
reservations	Lista de reservas do veículo.	list[dict]
utility	Objeto de métodos utilitários.	VehicleUtility

Métodos:

Nome	Parâmetros	Descrição
showInformations	self	Método para visualização de dados de conta.
showReservations	self	Método para visualizar todas as reservas efetuadas .
savingLoginData	self, dataFilePath(str)	Método para salvar novos dados gerados na opção "2 - CRIAR CONTA".
updateCredit	self, dataFilePath(str), value(float), operation(str)	Método para atualizar o valor de crédito da conta.
loadingData	self, dataFilePath(str), reservationsFilePath(str)	Método para carregar dados de conta (dados gerados e salvos anteriormente).
keepReservations	self, reservationsFilePath(str), newReservations(list[dict])	Método para salvar as novas reservas.

Os dados usados para preencher as variáveis são gerados no arquivo “main.py”, utilizando as bibliotecas **Faker** e **Random**.

4. VehicleUtility.py

Classe responsável por ter métodos de utilidade para o sistema, os métodos criados são:

Nome	Parâmetros	Descrição	Retorno
clearTerminal	-	Realiza a limpeza do terminal, com o objetivo de deixar o fluxo de exibição mais fluido e menos poluído.	-
endAnimation	self	Método que cria uma pequena animação de encerramento de programa.	-
defineRoute	self, origin(str), destination(str)	Método responsável por determinar os codinomes das cidades de origem e de destino, que são passados para o servidor, baseado nas respostas do usuário. Se as entradas forem válidas, retorna uma lista com os codinomes e caso alguma das entradas não corresponda com alguma das cidades da rota, o método retorna False e é executado novamente.	list[str] ou False
writeReplyBack	self, wrongActions (bool), repeat (bool)	Método utilizado para voltar para o início ou para encerrar o programa, dentro das opções de login(opções 1, 2, 3 e 4).	-
nomalizeName	self, fakeName(list[str])	Método para normalizar(tratar) a representação de caracteres especiais(ç) ou acentuações(á, é, à, è, õ, ã, ê, ô, â ...) na geração de nomes aleatórios(str).	genericName(list[str])
startAnimation	self	Método de exibição inicial no programa.	-

Na classe VehicleUtility há a importação das seguintes bibliotecas: **time**, **os** e **unicodedata**, para a estruturação da lógica dos métodos correspondentes.

5. VehicleClient.py

Classe responsável pela comunicação entre cliente Veículo e Servidores. A comunicação é baseada no protocolo MQTT(usando o broker Mosquitto) e no conceito de M2M. As variáveis e métodos da classe são:

Variáveis:

Nome	Descrição	Tipo
------	-----------	------

client	Objeto que estabelece a comunicação MQTT e usado para definir os seus parâmetros e métodos de execução.	mqtt.client
serverHOST	Host do server na comunicação MQTT.	str
serverPORT	Porta de rede na comunicação MQTT.	int
cost	Custo de serviço de todas reservas realizadas para a rota.	float
reservations	IDs de novas reservas serão adicionadas à conta do veículo.	list[str]
message	Variável usada para receber a resposta do servidor com as reservas. Posteriormente recebe um tipo json.	None
utility	Objeto de métodos utilitários.	VehicleUtility
vData	Dicionário estruturado para enviar as informações necessárias ao servidor ao realizar uma reserva. Ele é convertido em json posteriormente para ser enviado.	dict

Métodos:

Nome	Parâmetros	Descrição	Retorno
on_connect	self, client, userdata, flags, rc	Método de callback definido para ser executado ao estabelecer a conexão MQTT.	-
on_message	self, client, userdata, msg	Método de callback definido para tratar a mensagem/resposta na comunicação MQTT.	-

on_publish	self, client, userdata, mid	Método de callback para quando o client publica algo no tópico correspondente.	-
defineIP	self	Método para obter IP e determinar server baseado em um script. Utilizado para testes do projeto.	-

6. Dockerfile, docker-compose.yml e requirements.txt :

Dockerfile é o arquivo utilizado para estruturar a imagem do container do sistema e o **docker-compose.yml** é utilizado para estruturar um roteiro de execução desse container.

Dockerfile:

- A **imagem base** utilizada foi **"python:3.13.2"**;
- O arquivo principal a ser executado é **"main.py"**;
- Através da definição das tags **COPY requirements.txt** . e **RUN pip install --no-cache-dir -r requirements.txt** , as bibliotecas e dependências sejam instaladas no contêiner (flask, Faker e paho-mqtt).

docker_compose:

- O **nome do container** do cliente é **"vehicle_client"**;
- **"stdin_open"** e **"tty"** são definidos como **true** para permitir a execução do sistema de modo **interativo** (-it);
- A **rede** no modo **"network_mode: host"**;
- A porta definida é: **'8545:8545'**
- Tem como variáveis de ambiente:
 - **GANACHE_URL**: **"http://host.docker.internal:8545"** : Guarda a URL de acesso ao docker ganache;
 - **ACCOUNT_NUMBER** : Variável utilizada para determinar qual das 10 contas ganache predefinidas vai ser usada pelo veículo. **Essa variável deve ser alterada para cada novo veículo criado**, utilizando os índices de contas disponíveis.

Para executar o docker-compose.yml é preciso executá-lo com o seguinte roteiro:

→ **docker compose run --rm vehicle**

obs:

- O comando deve ser executado do diretório "**src/app/Vehicle**"
- O passo 2 não pode ser substituído por **docker compose up --build** pois, o container **vehicle_client** depende do modo interativo para funcionar corretamente, e o passo citado acima não permite esse modo;
- Tenha certeza que os outros containers (Servidores e Brokers Mosquitto) estejam sendo executados anteriormente para que a comunicação e a troca de informações ocorra corretamente;
- O passo 2 deve ser executado diretamente do diretório onde se encontra o arquivo **docker-compose.yml**.
- Ao encerrar o programa, por meio da tag **--rm**, o contêiner é removido.

7. Arquivos data.json, reservations.json e arquivos abi:

Arquivos utilizados para persistência de dados referentes à conta do veículo, o "data.json" é utilizado para salvar os dados referentes às informações tanto do proprietário como do próprio veículo, e o arquivo "reservations.json" é utilizado para salvar todas as reservas efetuadas na respectiva conta do veículo.

Arquitetura de Sistema dos Servidores:

O funcionamento da arquitetura dos servidores se manteve semelhante ao projeto anterior, do repositório "DistributedElectricCarsRechargingManager". Porém, as reservas, recargas e transações precisam ter seus blocos aprovados e inseridos na blockchain, do tipo "ledger", permitindo que os dados sejam imutáveis - apenas o "status" da informação pode ser alterado - e auditáveis.

Além disso, foi adicionada a entidade "Owner" que é responsável por executar o "deploy" dos contratos, autorizar contas Ganache de servidores a fazer operações na blockchain e liberar o pagamento das recargas a seus respectivos servidores e empresas.

Cenário de Funcionamento das Transações:

O cliente do veículo solicitará e receberá as reservas através dos tópicos MQTT;

O cliente do veículo depositará o pagamento das reservas, individualmente, através dos identificadores e dos valores delas na blockchain;

O cliente do veículo solicitará o início de uma sessão de carregamento, através do identificador da reserva, no tópico MQTT.

A solicitação do tópico MQTT será encaminhada para a blockchain, através de um dos servidores das empresas, que verificará se o pagamento foi depositado para a reserva em específico, se sim, a recarga será liberada.

O cliente do veículo solicitará a finalização de uma sessão de carregamento, através do identificador da reserva, no tópico MQTT.

A solicitação do tópico MQTT será encaminhada para a blockchain, através de um dos servidores das empresas, que liberará o pagamento da reserva da recarga encerrada para a carteira da devida empresa que prestou o serviço.

Passo-a-Passo de Execução da Blockchain e Servidores:

Cada entidade deve rodar em seu próprio container Docker e suas imagens podem ser geradas através do arquivo “app/docker-compose.yml” através do seguinte comando: “docker-compose build”.

Após gerar as imagens, é possível executar cada entidade através do seguinte comando: “docker-compose up [service]”

Ordem de execução dos containers das entidades: ganache, owner, mosquito, ecocharge, eflux, voltpoint.

Se os servidores forem executados em computadores diferentes, indique para cada um deles os endereços IP dos outros, editando as variáveis de ambiente nos arquivos “.env” de cada servidor.

Para o funcionamento adequado do sistema, o “deploy” dos contratos deve ser executado pelo cliente “owner”.

Tópicos MQTT “Subscriber” dos Servidores:

"vehicle/create_reservations/server", para agendar as reservas.

"vehicle/start_charging_session/server", para iniciar uma sessão de carregamento.

"vehicle/end_charging_session/server", para finalizar uma sessão de carregamento.

Tópicos MQTT “Publisher” dos Servidores:

"server/contracts_addresses/vehicle", para indicar os endereços dos contratos no Ganache.

"server/create_reservations/vehicle", para retornar os identificadores das reservas agendadas, ou erro no agendamento.

"server/start_charging_session/vehicle", para retornar sucesso ou falha, ao iniciar uma sessão de carregamento.

"server/end_charging_session/vehicle", para retornar sucesso ou falha, ao finalizar uma sessão de carregamento.

5. Resultados e Discussões

Ao final do desenvolvimento, tais conclusões podem ser feitas: O desenvolvimento dos sistemas clientes (Servidores e veículos) e broker foram realizadas e a comunicação entre eles foi bem sucedida. Entretanto, alguns requisitos do problema não foram devidamente tratados: A concorrência entre entidades e o tratamento de persistência e exibição das reservas que correspondem aos respectivos veículo

As demais funcionalidades e requisitos principais foram desenvolvidos e atendidos ao final do projeto, a comunicação e troca de pacotes de informações entre as entidades ocorreu de forma eficiente.

Referências:

VERONEZ, Fabricio. **Docker do zero ao compose: Parte 01**. YouTube, transmissão ao vivo. Disponível em: <https://www.youtube.com/live/GkMJJkWRgBQ>. Acesso em: 28 fev. 2025.

VERONEZ, Fabricio. **Docker do zero ao compose: Parte 02**. YouTube, transmissão ao vivo. Disponível em: https://www.youtube.com/live/KPb_ZqDTDyQ. Acesso em: 5 mar. 2025.

FERREIRA, Rafael. **Docker do zero ao compose: Parte 03**. Professor Lhama. YouTube, vídeo. Disponível em: <https://youtu.be/SxyHmEWtkuM>. Acesso em: 10 mar. 2025.

CARVALHO, Glauko. **Docker do zero ao compose: Parte 04**. YouTube, vídeo. Disponível em: <https://youtu.be/kP1kktlbUTs>. Acesso em: 18 mar. 2025.

FERSING, Pierre; LIGHT, Roger. **Paho-mqtt 2.1.0**. [S. l.], 29 abr. 2024. Disponível em: <https://pypi.org/project/paho-mqtt/#client>. Acesso em: 1 maio 2025.